JAVA DOCS

# Java 8 Features

Lambdas, Streams, Optional, and the Functional Programming Shift

Java Agentic AI — Knowledge Base Reference

# 1. Why Java 8 Mattered

Released in 2014, Java 8 was the largest change to the language since Java 5's generics. It introduced functional-style programming to a traditionally pure object-oriented language, primarily through lambda expressions, functional interfaces, the Stream API, and Optional — plus a new Date/Time API and default methods on interfaces (covered in the OOP Concepts doc).

# 2. Lambda Expressions

A lambda expression is an anonymous function — a concise way to implement a functional interface (an interface with exactly one abstract method) inline.

```
// Before Java 8
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Running"); }
};

// With lambda
Runnable r2 = () -> System.out.println("Running");

Comparator<String> byLength = (a, b) -> a.length() - b.length();
Comparator<String> byLength2 = (String a, String b) -> {
    return a.length() - b.length();     // block body needs explicit return
};

BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;
```

# 3. Functional Interfaces

| Interface | Method signature | Purpose |
|---|---|---|
| Function<T,R> | R apply(T t) | Transform T into R |
| Predicate<T> | boolean test(T t) | Boolean-valued test on T |
| Consumer<T> | void accept(T t) | Perform a side-effecting action on T |
| Supplier<T> | T get() | Produce/supply a value with no input |
| BiFunction<T,U,R> | R apply(T t, U u) | Two-arg transform |
| UnaryOperator<T> | T apply(T t) | Function<T,T> specialization |

```
@FunctionalInterface
interface Calculator {
    int calculate(int a, int b);
}
Calculator add = (a, b) -> a + b;
Calculator multiply = (a, b) -> a * b;

Predicate<Integer> isEven = n -> n % 2 == 0;
System.out.println(isEven.test(4));     // true
Predicate<Integer> isPositive = n -> n > 0;
```

```
Predicate<Integer> both = isEven.and(isPositive);
```

***Note:*** *@FunctionalInterface is optional but recommended -- it makes the compiler enforce exactly one abstract method, catching accidental additions early.*

## 4. Method References

```
list.forEach(System.out::println);         // reference to an instance method of a
particular object
list.stream().map(String::toUpperCase);     // reference to an instance method of an
arbitrary object
list.stream().map(Integer::parseInt);         // reference to a static method
Supplier<ArrayList<String>> ctor = ArrayList::new; // reference to a constructor
```

## 5. Optional

Optional is a container object that may or may not hold a non-null value, designed to make the possible absence of a value explicit in method signatures and reduce NullPointerException risk.

```
Optional<String> opt = Optional.ofNullable(getName());

opt.isPresent();                          // true/false
opt.get();                                  // throws NoSuchElementException if empty - use
cautiously
opt.orElse("default");                      // fallback value
opt.orElseGet(() -> computeDefault());     // lazy fallback
opt.orElseThrow(() -> new NoSuchElementException());

opt.ifPresent(name -> System.out.println(name));
opt.map(String::toUpperCase).ifPresent(System.out::println);

Optional<String> empty = Optional.empty();
```

***Note:*** *Optional is intended as a RETURN TYPE to signal 'this might be absent' -- avoid using it for fields, method parameters, or inside collections, which the JDK team explicitly discourages.*

## 6. Streams (Overview — full detail in Stream_API.pdf)

```
List<String> names = List.of("Alice", "Bob", "Charlie", "Dave");

List<String> result = names.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
```

## 7. New Date/Time API (java.time)

Replaces the mutable, thread-unsafe, poorly-designed java.util.Date and Calendar with an immutable, clear, ISO-8601-based API inspired by Joda-Time.

```
LocalDate date = LocalDate.of(2026, 7, 15);
LocalTime time = LocalTime.of(14, 30);
LocalDateTime dt = LocalDateTime.of(date, time);
ZonedDateTime zdt = ZonedDateTime.now(ZoneId.of("Asia/Kolkata"));

LocalDate tomorrow = date.plusDays(1);
Period period = Period.between(date, tomorrow);
Duration duration = Duration.ofMinutes(90);

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy");
String formatted = date.format(fmt);
```

| Old API | New API (java.time) |
|---|---|
| java.util.Date | LocalDate / LocalDateTime / Instant |
| java.util.Calendar | ZonedDateTime |
| Mutable, not thread-safe | Immutable, thread-safe by design |
| Month is 0-indexed (bug-prone) | Month is 1-indexed and enum-based |

## 8. Default and Static Methods in Interfaces

Covered fully in OOP_Concepts.pdf — Java 8 allowed interfaces to carry implementation via default methods (enabling backward-compatible API evolution, e.g. adding forEach() to Iterable without breaking every existing implementer) and static utility/factory methods.

## 9. Quick Interview Recap

- What is a functional interface? An interface with exactly one abstract method, targetable by a lambda expression or method reference.
- Why was Optional introduced? To make 'value may be absent' explicit in the type system and reduce unchecked NullPointerException risk in return types.
- Difference between map() and flatMap()? map() transforms each element 1-to-1; flatMap() transforms each element into a stream and flattens all resulting streams into one.
- Are lambdas objects? Yes — under the hood the JVM uses invokedynamic and generates a class implementing the target functional interface at runtime (not a plain anonymous inner class).
- Why is java.time better than java.util.Date? It's immutable (thread-safe by design), has a much clearer API, and fixes long-standing bugs like 0-indexed months.